



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks



Absolute Beginner to Specialization in Apache Spark and Azure Databricks



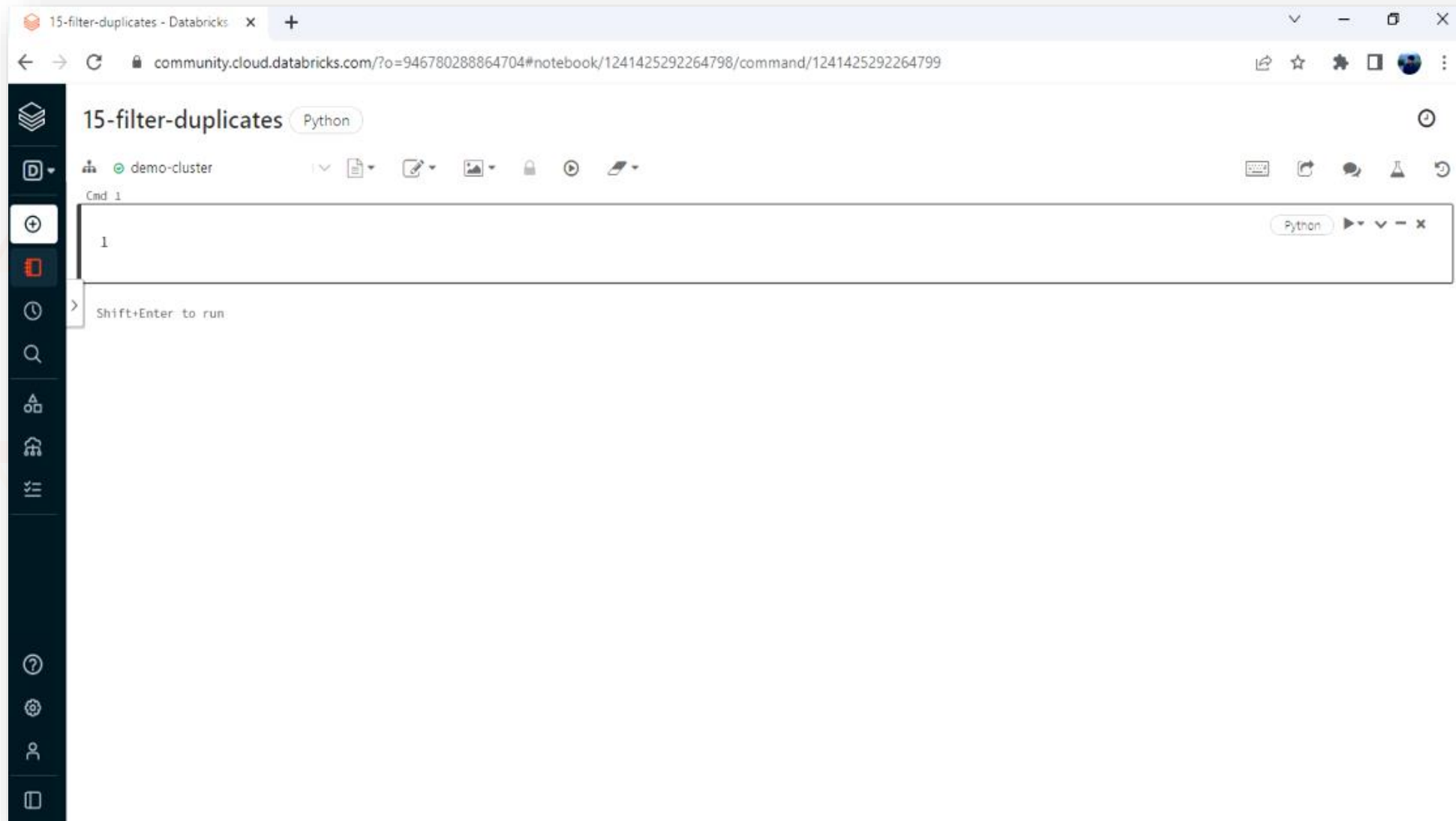
Filtering Records and Finding Unique Rows



In this chapter, we will talk about the following:

1. How to filter records from your Spark Dataframe
2. How to eliminate duplicates from your Dataframe

Go to your Databricks workspace and create a new notebook **(Reference : 15-filter-duplicates)**



Here we have a Dataframe. Now let's assume we want to filter records where the source column starts with Mum.

15-filter-duplicates

Python

demo-cluster

8

9

10

11

12

df = spark.createDataFrame(data_list) \

.toDF("id", "source", "destination", "distance")

display(df)

(3) Spark Jobs

	id	source	destination	distance
1	101	Mumbai	Goa	587
2	102	Mumbai	Bangalore	985
3	102	Mumbai	Bangalore	985
4	103	Delhi	Chennai	2208
5	104	Delhi	Chennai	2208
6	105	Bangalore	Kolkata	1868
7	105	Bangalore	Kolkata	1865

Showing all 7 rows.

Command took 6.19 seconds -- by prashant@scholarnest.com at 5/11/2022, 8:33:04 PM on demo-cluster

To filter records where the source column starts with Mum, we can use the following SQL command.

```
SELECT * FROM table_name  
WHERE source like 'Mum%'
```

How to do it in a dataframe?

Spark gives you two methods.

1. filter()
2. where()

The filter method allows you to filter records in the dataframe.
And the where() method is an alias of the filter() method.
So both are the same.

The filter condition is the same as the condition in a SQL expression. Then, I am chaining one more condition using the where method. There is no difference between filter() and where() because the where() method is just an alias of the filter() method.

```
> Cmd 2

1 df.filter("source like 'Mum%') \
2   .where("destination like 'Ban%') \
3   .show()

▶ (3) Spark Jobs

+---+-----+-----+-----+
| id|source|destination|distance|
+---+-----+-----+-----+
|102|Mumbai|  Bangalore|    985|
|102|Mumbai|  Bangalore|    985|
+---+-----+-----+-----+

Command took 0.79 seconds -- by prashant@scholarnest.com at 5/11/2022, 8:51:47 PM on demo-cluster
```

I used SQL conditions in the filter and where methods. And that's the most convenient method. But the filter method also allows you to specify conditions using functions and column API. This approach to writing expression is complex and confusing for SQL developers.

And there is no harm in taking the SQL approach and defining your expressions.

But learning the functional approach is good to have even though you rarely use it in practice. So this code takes the source column and applies column API to add the like() condition.

```
Cmd 3

1  from pyspark.sql.functions import col
2
3  df.filter(col("source").like("Mum%")) \
4  .where(col("destination").like("Ban%")) \
5  .show()

▶ (3) Spark Jobs

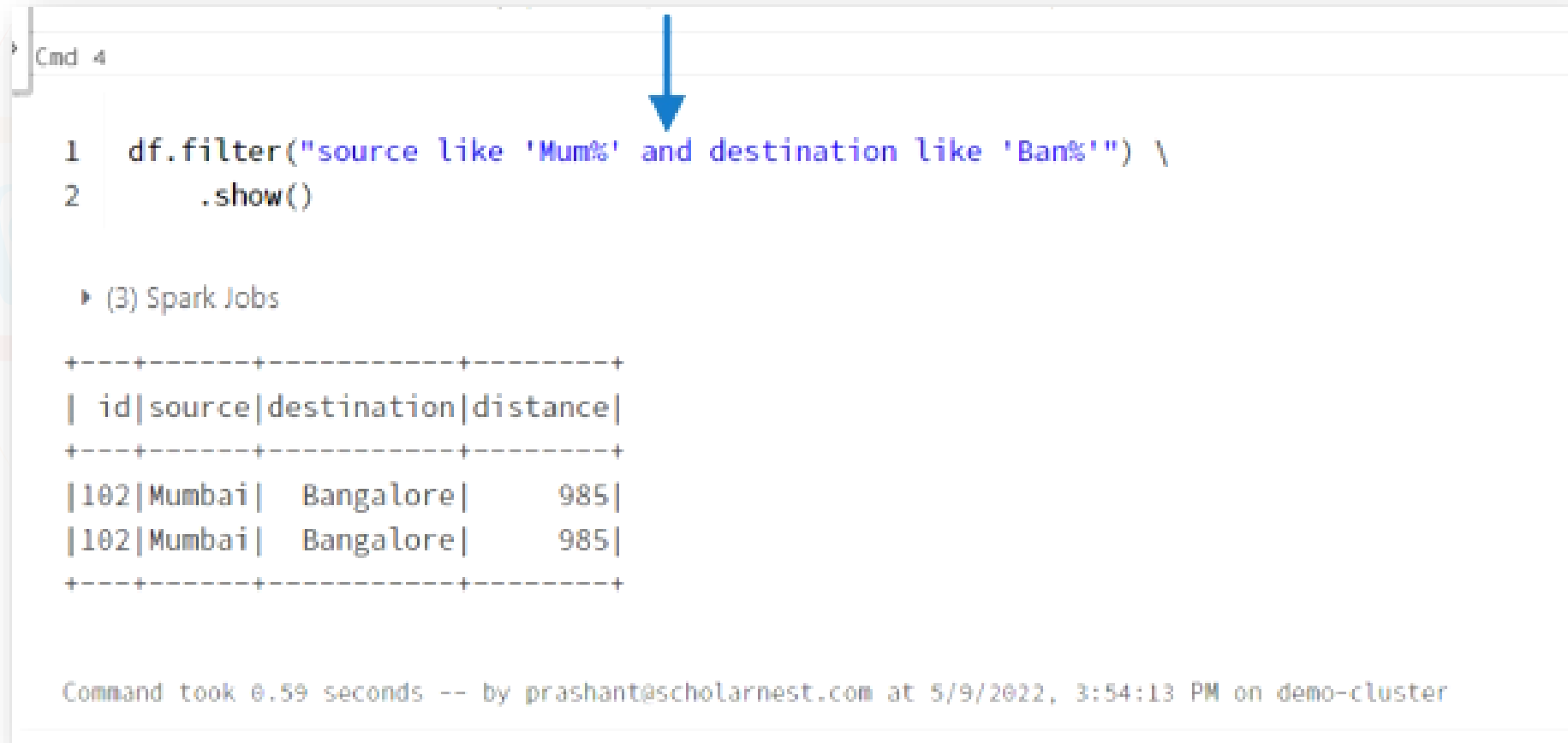
+---+-----+-----+-----+
| id|source|destination|distance|
+---+-----+-----+-----+
|102|Mumbai|  Bangalore|    985|
|102|Mumbai|  Bangalore|    985|
+---+-----+-----+-----+

Command took 0.58 seconds -- by prashant@scholarnest.com at 5/9/2022, 3:52:37 PM on demo-cluster
```

We can also use the previous example like this. I am explicitly using the AND operator between two conditions.

And this approach is more intuitive.

So we recommend using conditions explicitly.



```
Cmd 4
1 df.filter("source like 'Mum%' and destination like 'Ban%') \
2   .show()

▶ (3) Spark Jobs

+---+-----+-----+-----+
| id|source|destination|distance|
+---+-----+-----+-----+
|102|Mumbai|  Bangalore|    985|
|102|Mumbai|  Bangalore|    985|
+---+-----+-----+-----+

Command took 0.59 seconds -- by prashant@scholarnest.com at 5/9/2022, 3:54:13 PM on demo-cluster
```

Column APIs also allow using the and condition. The syntax is a little different as shown below. So we use a single ampersand between two conditions.

```
> Cmd 5
1  from pyspark.sql.functions import col
2
3  df.filter(col("source").like("Mum%") & col("destination").like("Ban%")) \
4      .show()

▶ (3) Spark Jobs

+---+-----+-----+-----+
| id|source|destination|distance|
+---+-----+-----+-----+
|102|Mumbai|  Bangalore|    985|
|102|Mumbai|  Bangalore|    985|
+---+-----+-----+-----+

Command took 0.50 seconds -- by prashant@scholarnest.com at 5/9/2022, 3:56:03 PM on demo-cluster
```

You can also use OR condition as shown below.

15-filter-duplicates Python

demo-cluster

id	source	destination	distance
101	Mumbai	Goa	587
102	Mumbai	Bangalore	985
102	Mumbai	Bangalore	985

Command took 0.50 seconds -- by prashant@scholarnest.com at 5/9/2022, 3:56:03 PM on demo-cluster

Cmd 6

```
1 df.filter("source like 'Mum%' or destination like 'Ban%') \
2     .show()
```

▶ (3) Spark Jobs

id	source	destination	distance
101	Mumbai	Goa	587
102	Mumbai	Bangalore	985
102	Mumbai	Bangalore	985

Command took 0.65 seconds -- by prashant@scholarnest.com at 5/9/2022, 3:57:07 PM on demo-cluster

We can change the ampersand to a pipe for using the OR condition in column API.

15-filter-duplicates Python

demo-cluster

id	source	destination	distance
101	Mumbai	Goa	587
102	Mumbai	Bangalore	985

Command took 0.65 seconds -- by prashant@scholarnest.com at 5/9/2022, 3:57:07 PM on demo-cluster

Cmd 7

```
1 from pyspark.sql.functions import col
2
3 df.filter(col("source").like("Mum%") | col("destination").like("Ban%")) \
4     .show()
```

▶ (3) Spark Jobs

id	source	destination	distance
101	Mumbai	Goa	587
102	Mumbai	Bangalore	985
102	Mumbai	Bangalore	985

Command took 0.52 seconds -- by prashant@scholarnest.com at 5/9/2022, 3:57:44 PM on demo-cluster

Here is the Dataframe shown below. And we have few problems for duplicity in this Dataframe. Let's try it one by one.

```
> Cmd 8
1 df.show()

▶ (3) Spark Jobs

+-----+-----+-----+
| id|   source|destination|distance|
+-----+-----+-----+
|101|  Mumbai|      Goa|    587|
|102|  Mumbai|Bangalore|    985|
|102|  Mumbai|Bangalore|    985|
|103|   Delhi|   Chennai|   2208|
|104|   Delhi|   Chennai|   2208|
|105|Bangalore|   Kolkata|   1868|
|105|Bangalore|   Kolkata|   1865|
+-----+-----+-----+

Command took 0.65 seconds -- by prashant@scholarnest.com at 5/11/2022, 9:26:10 PM on demo-cluster
Cmd 9
```

102 - Entire row is duplicate
105 - Record ID is duplicate
103/104 - ID is different, but other fields are duplicate

Here is the solution to the first problem. So one record is gone.

You can see only one record for 102.

The distinct() method is similar to the DISTINCT in SQL queries.

If your record is duplicated across all the columns, you can use the distinct() method.

```
> Cmd 9
1 df.distinct().show()

▶ (2) Spark Jobs

+-----+-----+-----+-----+
| id|   source|destination|distance|
+-----+-----+-----+-----+
|101|  Mumbai|      Goa|    587|
|102|  Mumbai|Bangalore|    985|
|103|   Delhi|   Chennai|   2208|
|104|   Delhi|   Chennai|   2208|
|105|Bangalore|   Kolkata|   1868|
|105|Bangalore|   Kolkata|   1865|
+-----+-----+-----+-----+

Command took 1.10 seconds -- by prashant@scholarnest.com at 5/11/2022, 9:26:42 PM on demo-cluster
```

We also have one more alternative for the first problem, the `dropDuplicates()` method. So the result of the `dropDuplicates()` and the `distinct()` is the same.

```
> Cmd 10  
  
1 df.dropDuplicates().show()  
  
▶ (2) Spark Jobs  
  
+-----+  
| id | source | destination | distance |  
+-----+  
|101| Mumbai | Goa | 587 |  
|102| Mumbai | Bangalore | 985 |  
|103| Delhi | Chennai | 2208 |  
|104| Delhi | Chennai | 2208 |  
|105| Bangalore | Kolkata | 1868 |  
|105| Bangalore | Kolkata | 1865 |  
+-----+  
  
Command took 0.68 seconds -- by prashant@scholarnest.com at 5/11/2022, 9:27:28 PM on demo-cluster
```

Now, look at the second problem. The record id 105 is duplicate, but other column values are different. This one is not a whole record duplication. So the distinct() method does not help. However, the dropDuplicates() method is more powerful. It can take a list of columns and eliminate duplicates based on those columns as shown below. So the drop dropDuplicates() will look at the id column and remove all records where the id is duplicate. And this guy solves two problems in one shot. It eliminated a duplicate of 102 and also 105. The 102 was a complete duplicate. That one is gone because the id was also duplicated. The 105 is fine because the id was duplicated.

```
> Cmd 11

1 df.dropDuplicates(["id"]).show()

▶ (2) Spark Jobs

id      source|destination|distance|
-----+-----+-----+
101     Mumbai|      Goa|      587|
102     Mumbai|Bangalore|      985|
103       Delhi|   Chennai|     2208|
104       Delhi|   Chennai|     2208|
105 Bangalore|   Kolkata|     1868|

Command took 1.31 seconds -- by prashant@scholarnest.com at 5/11/2022, 9:28:03 PM on demo-cluster

Cmd 12
```

Now let's go to the third problem.

103 and 104 are duplicates, but the ID is different.

How do we eliminate it? We can use the `dropDuplicates()` once again. But this time, we want to remove duplicates on all columns except the id column. Let me add another `dropDuplicates()` as shown below.

```
> Cmd 11

1 df.dropDuplicates(["id"]) \
2   .dropDuplicates(["source", "destination", "distance"]) \
3   .show()

▶ (3) Spark Jobs

+-----+
| id|  source|destination|distance|
+-----+
|102|  Mumbai|  Bangalore|    985|
|105|Bangalore|    Kolkata|   1868|
|101|  Mumbai|      Goa|    587|
|103|   Delhi|   Chennai|   2208|
+-----+
```

Command took 2.21 seconds -- by prashant@scholarnest.com at 5/11/2022, 9:28:56 PM on demo-cluster

So we learned two methods to remove duplicates.

1. `distinct()`

2. `dropDuplicates()`

The `distinct` method is similar to `DISTINCT` in the SQL, and it does not support complex scenarios.

The `dropDuplicates()` is more powerful. It takes a list of columns, compares records only using the given columns, and eliminates duplicates.



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com